Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ Инженерный бизнес и менеджмент
КАФЕДРА Промышленная логистика (ИБМ3)

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2
«Калькулятор. Функции на JavaScript»
ПО КУРСУ: РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЙ

НА ТЕМУ:
*Веб-приложение операционного учёта креперии:
меню с технологическими картами блюд*

Выполнил студент ИБМ3-54Б _____ А.К. Рак
(Подпись, дата) (И.О. Фамилия)

Проверил _____ _____
(Подпись, дата) (И.О. Фамилия)

2026 г.

ХОД РАБОТЫ

Задача: создание калькулятора, функции на JavaScript. Реализовать в калькуляторе индивидуальную операцию по теме; функция должна быть встроена в логику калькулятора без дополнительных окон и alert.

Описание проекта

К разметке и стилям лабораторной работы № 1 добавлен файл `script.js`, подключённый в секции `<head>`. Так как скрипт загружается раньше, чем браузер строит кнопки, весь код помещён в обработчик `window.onload`. В выпадающем списке режим SCI открывает панель дополнительных операций (рис. 1).



Рисунок 1 – Общий вид калькулятора в режиме SCI

Состояние калькулятора хранится в переменных: `a` и `b` – первое и второе число, `selectedOperation` – выбранная операция, `expressionResult` – результат, `justCalculated` – признак того, что на экране результат вычисления. Числа хранятся строками, так как вводятся по одной цифре, и превращаются в число унарным плюсом (`+a`) только при вычислении.

Ввод чисел

Все цифровые кнопки выбираются одним селектором атрибута `[id ^= "btn_digit_"]` и получают общий обработчик, который передаёт текст кнопки в функцию `onDigitButtonClicked` (рис. 2). Функция дописывает цифру в

текущее число (a до выбора операции, b после), не допускает вторую точку и ведущие нули и ограничивает длину ввода 14 символами.

script.js

```
47    function onDigitButtonClicked(digit) {
48      if (justCalculated && !selectedOperation) {
49        a = '';
50        justCalculated = false;
51      }
52
53      let current = getCurrent();
54
55      if (digit === '.') {
56        if (current.includes('.')) return;
57        if (current === '') current = '0';
58      } else {
59        if (current === '0') current = '';
60        if (digit === '000' && current === '') return;
61      }
62
63      if ((current + digit).length > 14) return;
64
65      setCurrent(current + digit);
66    }
⋮
148   document.querySelectorAll('[id ^= "btn_digit_"]').forEach(button => {
149     button.onclick = function () {
150       onDigitButtonClicked(button.innerHTML);
151     };
152   });
```

Рисунок 2 – Ввод цифры

Связь кнопки «=» с кодом и этапы сложения

Кнопке «=» в качестве обработчика клика назначена функция `calculate` (рис. 3, строка 159). Функция передаётся без скобок: обработчику нужна сама функция, а не результат её вызова. Функция проверяет наличие обоих чисел и операции, выбирает действие оператором `switch`, округляет результат до 10 знаков и записывает его в a, чтобы с ним можно было продолжить вычисления.

script.js

```
68   function calculate() {
69     if (a === '' || b === '' || !selectedOperation) return;
70
71     switch (selectedOperation) {
72       case '+':
73         expressionResult = (+a) + (+b);
74         break;
75       case '-':
76         expressionResult = (+a) - (+b);
77         break;
78       case 'x':
79         expressionResult = (+a) * (+b);
80         break;
81       case '/':
82         if (+b === 0) {
83           showError();
84           return;
85         }
86         expressionResult = (+a) / (+b);
87         break;
88     }
89
90     expressionResult = parseFloat(expressionResult.toFixed(10));
91     a = expressionResult.toString();
92     b = '';
93     selectedOperation = null;
94     show(a);
95     justCalculated = true;
96   }
97
98   function chooseOperation(operation) {
99     if (a === '') return;
100    if (b !== '' && selectedOperation) {
101      calculate();
102      if (a === '') return;
103    }
104    selectedOperation = operation;
105  }
154  document.getElementById('btn_op_plus').onclick = function () { chooseOperation('+'); };
155  document.getElementById('btn_op_minus').onclick = function () { chooseOperation('-'); };
156  document.getElementById('btn_op_mult').onclick = function () { chooseOperation('x'); };
157  document.getElementById('btn_op_div').onclick = function () { chooseOperation('/'); };
158
159  document.getElementById('btn_op_equal').onclick = calculate;
```

Рисунок 3 – Функции calculate и chooseOperation, обработчики операций

Сложение чисел 12 и 7 выполняется по этапам:

1) нажатие «1» и «2»: обработчик цифровых кнопок вызывает `onDigitButtonClicked`, операция не выбрана, поэтому цифры дописываются в строку `a` = "12", результат выводится функцией `show`;
2) нажатие «+»: функция `chooseOperation("+")` записывает операцию в `selectedOperation`;

3) нажатие «7»: операция выбрана, поэтому цифра записывается во второе число `b` = "7";
4) нажатие «=»: вызывается `calculate`; оператор `switch` выбирает ветку `case "+"`, строки превращаются в числа и складываются: `(+a) + (+b)` = 19;
5) результат записывается в `a` = "19", `b` очищается, операция сбрасывается, на экран выводится 19.

Если после ввода «12 + 7» повторно нажать «+», функция `chooseOperation` сначала вычисляет промежуточный итог и только затем запоминает новую операцию (рис. 4). Так реализованы накапливаемые сложение и вычитание (задания 9 и 10).

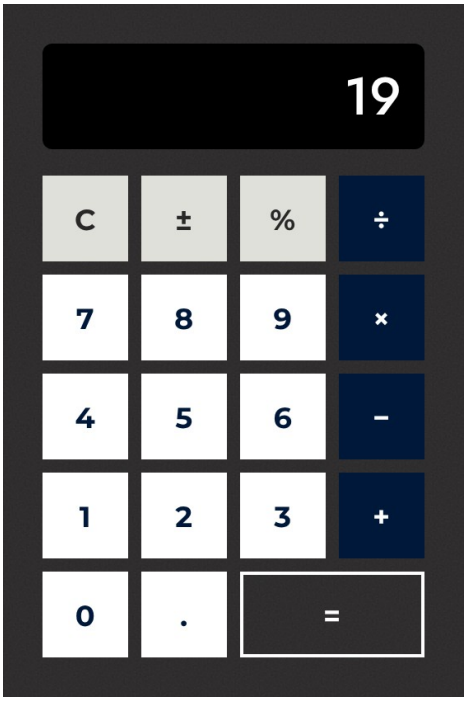


Рисунок 4 – Промежуточный итог 19 после ввода «12 + 7 +»

Индивидуальная операция «Цена»

Индивидуальная операция по теме – расчёт цены продажи блюда по его себестоимости. В креперии цену позиции меню назначают так, чтобы доля себестоимости в цене (фудкост) не превышала целевого значения, обычно 25–35 %. Кнопка «Цена» принимает себестоимость блюда и выдаёт цену при целевом фудкосте 30 %, округлённую вверх до 10 рублей:

$$Цена = \lceil Себестоимость / 0{,}3 / 10 \rceil \cdot 10.$$

Реализация приведена на рисунке 5. Целевой фудкост вынесен в константу `targetFoodCost`. Функция `calcCrepePrice` выполняет расчёт, а обработчик кнопки передаёт её в общую функцию `applyUnary`, которая применяет операцию к текущему числу и при некорректном результате выводит ERROR.

Поэтому операция встроена в логику калькулятора: её можно применять и к первому, и ко второму числу выражения.

script.js

```
13    const ledColors = ['#FFFFFF', '#E0E0DB', '#E8A33D'];
14    const targetFoodCost = 0.3;
      ⋮
107   function applyUnary(operation) {
108     const current = getCurrent();
109     if (current === '') return;
110
111     const result = operation(+current);
112
113     if (!Number.isFinite(result)) {
114       showError();
115       return;
116     }
117
118     setCurrent(String(parseFloat(result.toFixed(10))));
119     if (!selectedOperation) justCalculated = true;
120   }
121
122   function calcCrepePrice(crepeCost) {
123     if (crepeCost <= 0) return NaN;
124     return Math.ceil(crepeCost / targetFoodCost / 10) * 10;
125   }
      ⋮
202   document.getElementById('btn_op_crepe_price').onclick = function () {
203     applyUnary(calcCrepePrice);
204   };
```

Рисунок 5 – Операция «Цена»: константа, функции applyUnary и calcCrepePrice, обработчик кнопки

Пример: для блинчика с себестоимостью 250 ₽ операция выдаёт цену 840 ₽ (рис. 6); фудкост составит 250 / 840 ≈ 29,8 %. Для себестоимости 0 или отрицательной операция выводит ERROR.

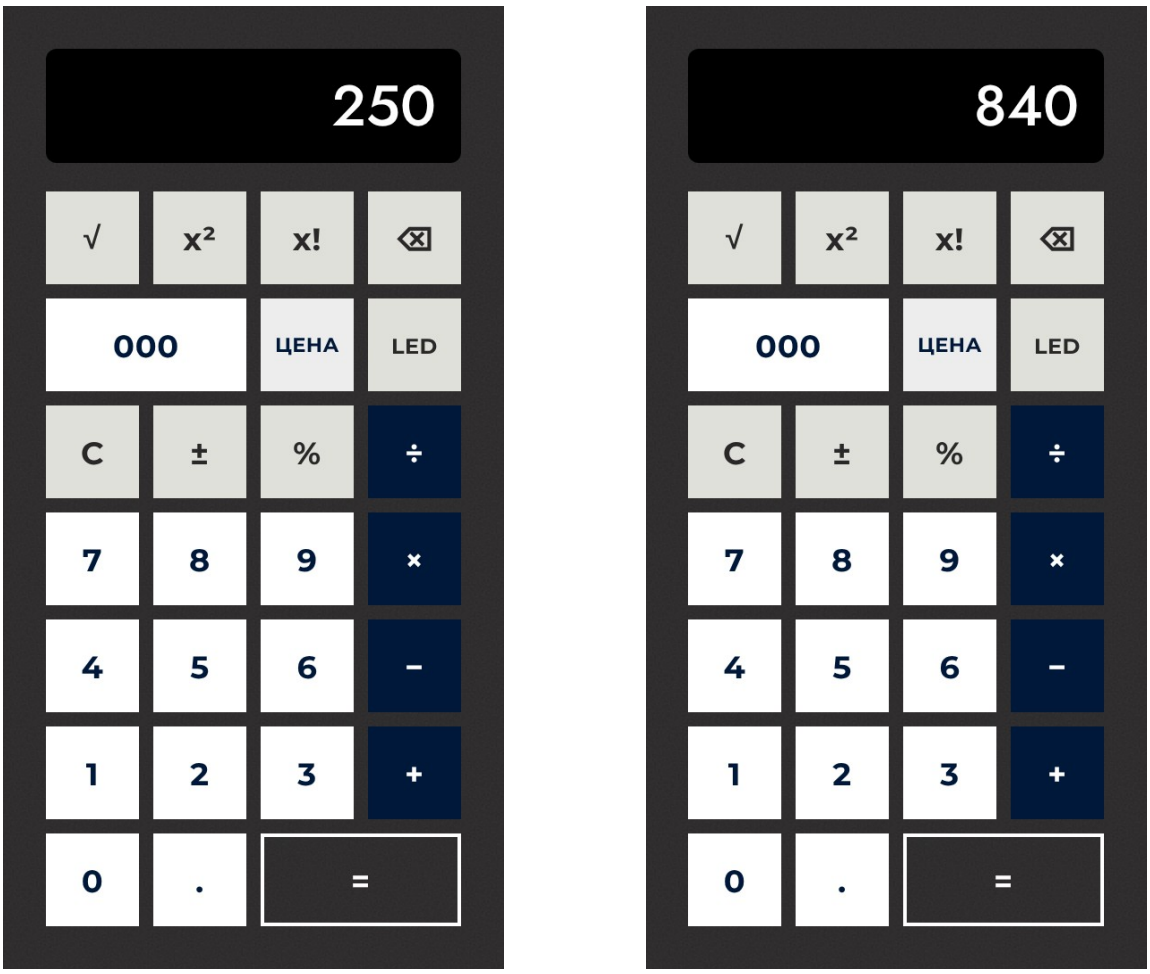


Рисунок 6 – Себестоимость 250 до (слева) и после (справа) нажатия «Цена»

Остальные функции

Процент, квадратный корень, квадрат и факториал выполняются той же функцией `applyUnary`; факториал вычисляется циклом `for` и определён только для целых неотрицательных чисел. Смена знака и стирание последнего символа работают со строкой текущего числа (рис. 7).

script.js

```
127   function factorial(n) {
128     if (n < 0 || !Number.isInteger(n)) return NaN;
129     let result = 1;
130     for (let i = 2; i <= n; i++) {
131       result *= i;
132     }
133     return result;
134   }
      ⋮
170   document.getElementById('btn_op_percent').onclick = function () {
171     applyUnary(x => x / 100);
172   };
173
174   document.getElementById('btn_op_sqrt').onclick = function () {
175     applyUnary(Math.sqrt);
176   };
177
178   document.getElementById('btn_op_square').onclick = function () {
179     applyUnary(x => x * x);
180   };
181
182   document.getElementById('btn_op_fact').onclick = function () {
183     applyUnary(factorial);
184   };
185
186   document.getElementById('btn_op_sign').onclick = function () {
187     const current = getCurrent();
188     if (current === '' || +current === 0) return;
189     if (current.startsWith('-')) {
190       setCurrent(current.slice(1));
191     } else {
192       setCurrent('-' + current);
193     }
194   };
195
196   document.getElementById('btn_op_back').onclick = function () {
197     let shorter = getCurrent().slice(0, -1);
198     if (shorter === '-') shorter = '';
199     setCurrent(shorter);
200   };
```

Рисунок 7 – Факториал и обработчики операций над одним числом

При делении на ноль, извлечении корня из отрицательного числа и факториале дробного числа выводится ERROR (рис. 8), а выполнение скрипта не прерывается.

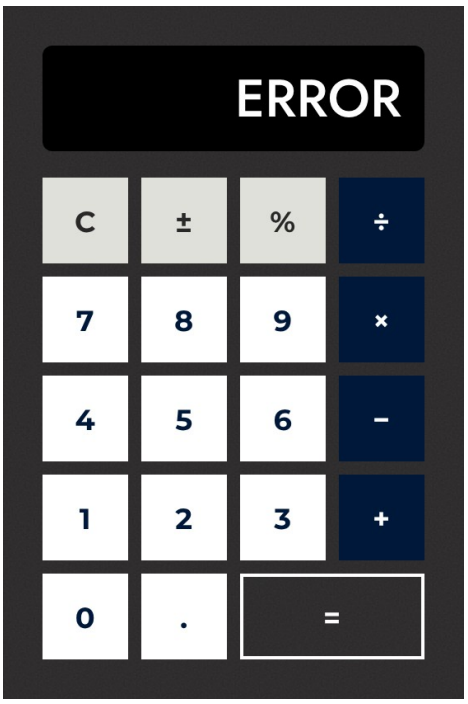


Рисунок 8 – Вывод ERROR при делении на ноль

Обработчики смены темы, режима SCI и цвета поля вывода приведены на рисунке 9. Тема и режим переключаются классами (`dark` у `<body>`, `sci` у корпуса), а отображение определяет CSS. Кнопка LED перебирает цвета поля вывода по кругу с помощью остатка от деления (рис. 10).

script.js

```
136   document.querySelector('.theme-toggle').onclick = function () {
137     document.body.classList.toggle('dark');
138   };
139
140   modeSelect.onchange = function () {
141     if (modeSelect.value === 'SCI') {
142       device.classList.add('sci');
143     } else {
144       device.classList.remove('sci');
145     }
146   };
      ⋮
206   document.getElementById('btn_op_led').onclick = function () {
207     ledIndex = (ledIndex + 1) % ledColors.length;
208     outputElement.style.color = ledColors[ledIndex];
209   };
```

Рисунок 9 – Обработчики темы, режима и цвета поля вывода



Рисунок 10 – Поле вывода после смены цвета кнопкой LED

Выполнение заданий

Выполнение заданий для самостоятельной проработки приведено в таблице 1.

Таблица 1 – Выполнение заданий лабораторной работы № 2

№	Задание	Реализация
1	Смена знака +/-	добавление или удаление минуса в строке числа
2	Вычисление процента	деление на 100 через applyUnary
3	Стирание введённой цифры	кнопка ⌫, метод slice(0, -1)
4	Смена цвета фона	кнопка «Тема», класс dark
5	Квадратный корень	Math.sqrt; для отрицательных – ERROR
6	Возведение в квадрат	умножение числа на само себя
7	Факториал	цикл for; для дробных и отрицательных – ERROR
8	Кнопка 000	кнопка из группы btn_digit_
9	Накапливаемое сложение	chooseOperation: промежуточный итог

№	Задание	Реализация
10	Накапливаемое вычитание	chooseOperation, аналогично заданию 9
11	Смена цвета окна вывода	кнопка LED, style.color
12	Индивидуальная операция по теме	кнопка «Цена»: цена по себестоимости при фудкосте 30 %

ВЫВОД

В ходе работы освоены основы JavaScript: обработка событий, хранение состояния, функции, оператор switch, циклы, преобразование строк в числа и обработка ошибочных ситуаций. Реализована логика калькулятора и выполнены все 12 заданий. Добавлена индивидуальная операция «Цена» – расчёт цены блюда по себестоимости при целевом фудкосте 30 %, встроенная в общую логику калькулятора без дополнительных окон. Исправлены ошибки базовой реализации из методических указаний: ввод после «=», деление на ноль и погрешность дробных чисел.

Исходный код размещён в репозитории GitHub: https://github.com/rakulasi/webchi-labs.

СПИСОК ЛИТЕРАТУРЫ

1. Курс «JavaScript»: лекции и методические указания [Электронный ресурс] // GitHub. – URL: https://github.com/iu5git/JavaScript (дата обращения: 28.10.2026).
2. MDN Web Docs. Справочник по HTML, CSS и JavaScript [Электронный ресурс]. – URL: https://developer.mozilla.org/ru/ (дата обращения: 28.10.2026).
3. Кантор И. Современный учебник JavaScript [Электронный ресурс]. – URL: https://learn.javascript.ru (дата обращения: 28.10.2026).
4. HTMLBook. Справочник по HTML и CSS [Электронный ресурс]. – URL: http://htmlbook.ru (дата обращения: 28.10.2026).
5. Google Fonts: Montserrat, Jost [Электронный ресурс]. – URL: https://fonts.google.com (дата обращения: 28.10.2026).
6. Breizh Café. Crêperie bretonne [Электронный ресурс]. – URL: https://www.breizhcafe.com (дата обращения: 28.10.2026).